

xBrain Backend – Sprint Report

1. Project Overview

xBrain is an educational platform designed to help users build their professional profiles, explore specializations, and earn points and badges. This sprint introduces three major new feature modules (Q&A, Posts, Comments), a generic file-attachment system, immediate-logout authentication, and cloud-native production infrastructure.

Component	Technology
Framework	Django 5.2 + Django REST Framework
Database	PostgreSQL (Azure Flexible Server)
Cache & Token Blacklist	Redis (Azure Cache for Redis)
Media Storage	Azure Blob Storage
Hosting	Azure App Service (Linux)
Authentication	JWT (SimpleJWT) + custom blacklist-aware layer
Documentation	Swagger UI (drf-spectacular / OpenAPI 3.0)

Live API URL: <https://xbrain-backend-chbfe7hscpbqergn.francecentral-01.azurewebsites.net>

Swagger Documentation: <https://xbrain-backend-chbfe7hscpbqergn.francecentral-01.azurewebsites.net/api/docs/>

2. New Features

2.1 Q&A System

A complete question-and-answer module that allows users to post questions tagged with specializations, receive answers, and engage in threaded discussion through one-level replies.

Step	Endpoint	Description
Create Question	POST /api/questions/	Ask a new question, optionally tagged with specializations and attachments.
List Questions	GET /api/questions/	Browse all questions with author, resolution status, and counts.
Question Detail	GET / PATCH / DELETE /api/questions/{id}/	View, edit, or delete a question (edits restricted to author).
Resolve	POST /api/questions/{id}/resolve/	Mark the question as resolved (author only).
Unresolve	POST /api/questions/{id}/unresolve/	Reopen a resolved question.
Post Answer	POST /api/questions/{id}/answers/	Submit an answer to a question.
Answer Detail	GET / PATCH / DELETE /api/answers/{id}/	View, edit, or delete an answer.
Reply to Answer	POST /api/answers/{id}/replies/	Reply to an existing answer (depth limited to 1 level).

The reply-depth invariant is enforced at the model layer to prevent deeply nested threads:

```
# api/models.py – Answer model
class Answer(models.Model):
    parent_answer = models.ForeignKey(
        'self', on_delete=models.CASCADE,
        null=True, blank=True, related_name='replies',
    )

    def clean(self):
        if self.parent_answer and self.parent_answer.parent_answer_id is not None:
            raise ValidationError('Replies cannot have replies – depth limit is 1.')
```

2.2 Posts Module with Reactions

A knowledge-sharing posts module where users can publish content and react with likes or dislikes. A user may have at most one active reaction per post; toggling one removes the other.

Step	Endpoint	Description
Create Post	POST /api/posts/	Publish a new post with text and optional attachments.
List Posts	GET /api/posts/	Browse the feed with reaction counts and author info.
Post Detail	GET / PATCH / DELETE /api/posts/{id}/	View, edit, or delete a post (author only).
Like Post	POST /api/posts/{id}/like/	Toggle like reaction.
Dislike Post	POST /api/posts/{id}/dislike/	Toggle dislike reaction.

The reaction toggle uses a database-level transaction and a `SELECT ... FOR UPDATE` lock to safely handle the three possible cases — first reaction, switching reaction type, or removing — with no race conditions:

```
# api/views.py – PostReaction toggle
def _toggle(self, request, pk):
    post = get_object_or_404(Post, pk=pk)
    with transaction.atomic():
        existing = (
            PostReaction.objects
            .select_for_update()
            .filter(user=request.user, post=post)
            .first()
        )
        if existing is None:
            PostReaction.objects.create(
                user=request.user, post=post, reaction=self.target_reaction,
            )
        elif existing.reaction == self.target_reaction:
            existing.delete()          # toggle off
        else:
            existing.reaction = self.target_reaction
            existing.save(update_fields=['reaction', 'updated_at']) # switch
```

2.3 Comments with Threaded Replies

A threaded comment system on posts. Comment deletion follows a dual-authority rule: either the comment author **or** the post owner can delete a comment, while edits remain author-only.

Step	Endpoint	Description
Create Comment	POST /api/posts/{post_id}/comments/	Add a comment to a post.
List Comments	GET /api/posts/{post_id}/comments/	Browse comments on a post.
Comment Detail	GET / PATCH / DELETE /api/comments/{id}/	View, edit (author), or delete (author or post owner).
Reply to Comment	POST /api/comments/{id}/replies/	Add a threaded reply on a comment.
List Replies	GET /api/comments/{id}/replies/	Browse replies on a comment.

The dual-authority delete rule is encoded as a reusable DRF permission class:

```
# api/permissions.py
class IsCommentDeletable(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        if request.method == 'DELETE':
            return (
                obj.author_id == request.user.id
                or obj.post.author_id == request.user.id
            )
        return obj.author_id == request.user.id # PATCH – author only
```

2.4 Multi-type Attachments

A generic file-attachment system that allows uploading images, videos, audio recordings, and PDFs to questions, answers, or posts. The same model serves all parent types via Django's `ContentType` framework, and uploads are stored on Azure Blob Storage.

Each upload is classified by MIME type and size-validated:

Type	Allowed formats	Maximum size
Image	JPEG, PNG, WEBP	5 MB
Video	MP4, QuickTime	50 MB
Audio	MP3, M4A/AAC, OGG, WebM	15 MB
PDF	PDF	10 MB

The validation rules live in a single configuration dictionary, applied uniformly across every parent type:

```
# api/utils.py
ATTACHMENT_LIMITS = {
    'image': {
        'mime_types': ['image/jpeg', 'image/png', 'image/webp'],
        'max_bytes': 5 * 1024 * 1024,
    },
    'video': {
        'mime_types': ['video/mp4', 'video/quicktime'],
        'max_bytes': 50 * 1024 * 1024,
    },
    'audio': {
        'mime_types': [
            'audio/mpeg', 'audio/mp4', 'audio/aac',
            'audio/ogg', 'audio/webm',
        ],
        'max_bytes': 15 * 1024 * 1024,
    },
    'pdf': {
        'mime_types': ['application/pdf'],
        'max_bytes': 10 * 1024 * 1024,
    },
}

MAX_ATTACHMENTS_PER_PARENT = 4

def classify_and_validate_attachment(uploaded_file):
    mime = (uploaded_file.content_type or '').lower()
    for kind, limits in ATTACHMENT_LIMITS.items():
        if mime in limits['mime_types']:
            if uploaded_file.size > limits['max_bytes']:
                max_mb = limits['max_bytes'] // (1024 * 1024)
                raise DRFValidationError(
                    f'{kind.capitalize()} too large (max {max_mb} MB).'
                )
            return kind, mime
    raise DRFValidationError(f'Unsupported file type: {mime or "unknown"}.')

```

A maximum of 4 attachments can be associated with any single parent resource. Files are uploaded inline as part of the parent's POST request via `multipart/form-data`.

Step	Endpoint	Description
Delete Attachment	DELETE /api/attachments/{id}/	Remove an attachment (owner only).

2.5 User Certificates

A full certificate management module on the user profile. Users can list, add, and remove their own certificates and view another user's public certificate list.

Step	Endpoint	Description
List Own Certificates	GET /api/users/me/certificates/	Browse the current user's certificates.
Add Certificate	POST /api/users/me/certificates/	Add a new certificate (title, issuer, issue date, URL).
Delete Certificate	DELETE /api/users/me/certificates/{id}/	Remove an own certificate.
View Public Certificates	GET /api/users/{user_id}/certificates/	List another user's certificates.

The Certificate model captures four professional attributes and links back to a user with cascade delete:

```
# api/models.py – Certificate model
class Certificate(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user = models.ForeignKey(
        'User', on_delete=models.CASCADE,
        related_name='certificates', db_index=True,
    )
    title = models.CharField(max_length=200)
    issuer = models.CharField(max_length=200)
    issue_date = models.DateField()
    certificate_url = models.URLField(max_length=500)
```

2.6 Logout with Immediate Token Invalidation

A logout endpoint that invalidates the user's session in real time. Unlike standard JWT logout — which only blacklists refresh tokens — this implementation also blacklists the current access token by recording its JTI (JWT ID) in Redis. The token is rejected on the very next request, not at its natural expiry.

Step	Endpoint	Description
Logout	POST /api/auth/logout/	Invalidate the user's access and refresh tokens immediately.

```
# api/views.py – LogoutView
class LogoutView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        refresh = request.data.get('refresh')
        if not refresh:
            return Response({'refresh': ['This field is required.']},
                            status=status.HTTP_400_BAD_REQUEST)

        try:
            RefreshToken(refresh).blacklist()
        except Exception:
            return Response({'refresh': ['Invalid or expired refresh token.']},
                            status=status.HTTP_400_BAD_REQUEST)

        # Access token's JTI is also added to a Redis blacklist key
        # so the current access token is rejected on the next request.
```

2.7 Custom JWT Authentication Layer

A custom DRF authentication class — `BlacklistAwareJWTAuthentication` — that extends SimpleJWT's authentication to check the Redis-backed access-token blacklist on every request. This class is configured as the default authentication class for the entire API, so every protected endpoint enforces immediate logout consistently.

```
# api/authentication.py
from django.core.cache import cache
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework_simplejwt.exceptions import InvalidToken

ACCESS_BLACKLIST_KEY_PREFIX = 'jwt_access_blacklist_'

def access_blacklist_key(jti: str) -> str:
    return f'{ACCESS_BLACKLIST_KEY_PREFIX}{jti}'

class BlacklistAwareJWTAuthentication(JWTAuthentication):
    def get_validated_token(self, raw_token):
        validated_token = super().get_validated_token(raw_token)
        jti = validated_token.get('jti')
        if jti and cache.get(access_blacklist_key(jti)):
            raise InvalidToken('Access token has been logged out.')
        return validated_token
```

2.8 Object-level Permission Classes

Three reusable permission classes that enforce ownership and moderation rules across the API:

Permission Class	Rule
IsAuthorOrReadOnly	Read for any authenticated user; write only for the object's author.
IsQuestionAuthor	Restricts question resolve/unresolve to the question's author.
IsCommentDeletable	Author OR post owner can delete a comment; only the author can edit.

```
# api/permissions.py
class IsAuthorOrReadOnly(BasePermission):
    """Read for anyone (subject to view-level auth); write only for the author."""
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.author_id == request.user.id

class IsQuestionAuthor(BasePermission):
    """Only the question's author can resolve / unresolve it."""
    def has_object_permission(self, request, view, obj):
        return obj.author_id == request.user.id
```

2.9 Azure Blob Storage Integration

Media storage migrated from the App Service's local disk to **Azure Blob Storage**. All user-uploaded files — profile images, question/answer/post attachments — are now stored durably in the cloud and served directly from Azure Storage URLs.

The storage backend is environment-driven: production uses Azure Blob, local development continues to use the filesystem. No code changes are required to switch between them — only an environment variable toggle (`USE_AZURE_STORAGE`).

```
# xBrain/settings.py
USE_AZURE_STORAGE = config('USE_AZURE_STORAGE', default=False, cast=bool)

STORAGES = {
    "default": {
        "BACKEND": (
            "storages.backends.azure_storage.AzureStorage"
            if USE_AZURE_STORAGE
            else "django.core.files.storage.FileSystemStorage"
        ),
    },
    "staticfiles": {
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}

if USE_AZURE_STORAGE:
    AZURE_ACCOUNT_NAME = config('AZURE_ACCOUNT_NAME')
    AZURE_ACCOUNT_KEY = config('AZURE_ACCOUNT_KEY')
    AZURE_CONTAINER = config('AZURE_CONTAINER', default='media')
```


Configuration steps performed during this sprint: - Provisioned an Azure Storage Account in the same region as the App Service. - Created a public-read blob container named `media`. - Installed and configured the `django-storages[azure]` package. - Added `AZURE_ACCOUNT_NAME`, `AZURE_ACCOUNT_KEY`, and `AZURE_CONTAINER` to App Service application settings.

2.10 Redis Cache Backend

The previous in-memory cache was replaced with **Redis** (Azure Cache for Redis). Redis backs the OTP store, login attempt counters, password reset tokens, and the access-token blacklist. This change makes the API safe to run with multiple Gunicorn workers and ready for horizontal scaling.

```
# xBrain/settings.py
REDIS_URL = config('REDIS_URL', default=None)
if REDIS_URL:
    CACHES = {
        'default': {
            'BACKEND': 'django.core.cache.backends.redis.RedisCache',
            'LOCATION': REDIS_URL,
            'TIMEOUT': 300,
        }
    }
else:
    # Local-dev fallback (single-worker only)
    CACHES = {
        'default': {
            'BACKEND': 'django.core.cache.backends.locmem.LocMemCache',
            'LOCATION': 'xbrain-default',
            'TIMEOUT': 300,
        }
    }
```

3. New API Endpoints

3.1 Q&A

Method	Endpoint	Description
GET	/api/questions/	List questions
POST	/api/questions/	Create a question
GET	/api/questions/{id}/	Question detail
PATCH	/api/questions/{id}/	Edit a question (author only)
DELETE	/api/questions/{id}/	Delete a question (author only)
POST	/api/questions/{id}/resolve/	Mark as resolved
POST	/api/questions/{id}/unresolve/	Reopen a question
GET	/api/questions/{id}/answers/	List answers on a question
POST	/api/questions/{id}/answers/	Post an answer
GET	/api/answers/{id}/	Answer detail
PATCH	/api/answers/{id}/	Edit an answer (author only)
DELETE	/api/answers/{id}/	Delete an answer (author only)
GET	/api/answers/{id}/replies/	List replies on an answer
POST	/api/answers/{id}/replies/	Reply to an answer (depth limited to 1)

3.2 Posts

Method	Endpoint	Description
GET	/api/posts/	List posts feed
POST	/api/posts/	Create a post
GET	/api/posts/{id}/	Post detail
PATCH	/api/posts/{id}/	Edit a post (author only)
DELETE	/api/posts/{id}/	Delete a post (author only)
POST	/api/posts/{id}/like/	Toggle like reaction
POST	/api/posts/{id}/dislike/	Toggle dislike reaction

3.3 Comments

Method	Endpoint	Description
GET	/api/posts/{post_id}/comments/	List comments on a post
POST	/api/posts/{post_id}/comments/	Comment on a post
GET	/api/comments/{id}/	Comment detail
PATCH	/api/comments/{id}/	Edit a comment (author only)
DELETE	/api/comments/{id}/	Delete a comment (author or post owner)
GET	/api/comments/{id}/replies/	List replies on a comment
POST	/api/comments/{id}/replies/	Reply to a comment

3.4 Attachments

Method	Endpoint	Description
DELETE	/api/attachments/{id}/	Delete an attachment (owner only)

3.5 Certificates

Method	Endpoint	Description
GET	/api/users/me/certificates/	List own certificates
POST	/api/users/me/certificates/	Add a certificate
DELETE	/api/users/me/certificates/{id}/	Delete an own certificate
GET	/api/users/{user_id}/certificates/	Public list of a user's certificates

3.6 Auth (New)

Method	Endpoint	Description
POST	/api/auth/logout/	Logout and invalidate tokens (immediate)

4. New Database Models

Six new models were introduced this sprint, across four migration files:

Model	Purpose
Question	A user-submitted question, optionally tagged with specializations
Answer	An answer or threaded reply on a question
Post	A knowledge-sharing post
PostReaction	A user's like or dislike on a post (one per user per post)
Comment	A threaded comment or reply on a post
Attachment	A generic file attachment that can hang off a Question, Answer, or Post

All new models use UUID primary keys and include database indexes optimized for the most common query patterns.

The `PostReaction` model enforces the one-reaction-per-user-per-post rule at the database level via a `unique_together` constraint:

```
# api/models.py – PostReaction
class PostReaction(models.Model):
    user = models.ForeignKey('User', on_delete=models.CASCADE, related_name='post_reactions')
    post = models.ForeignKey('Post', on_delete=models.CASCADE, related_name='reactions')
    reaction = models.CharField(max_length=10, choices=REACTION_CHOICES)
    # ... timestamps ...

    class Meta:
        unique_together = ('user', 'post')
        indexes = [models.Index(fields=['post', 'reaction'], name='idx_pr_post_reaction')]
```

5. New Tests

Test coverage was expanded for every new feature area. Six new test modules were added:

Module	Coverage
tests_logout.py	Logout endpoint and access-token blacklist
tests_qa.py	Questions, answers, replies, resolve/unresolve, permissions
tests_posts.py	Posts and like/dislike reactions
tests_comments.py	Comment threading and delete-permission rules
tests_certificates.py	Certificate CRUD and public listing
tests_attachments.py	Multi-type file upload validation and storage